

PRACTICAL: 1**Date:** / /**TITLE:** Find probability of coin tossing events for multiple coins.

- Accept no. of coins from user.
- Accept value of multiple tossing of each coin.
- Display occurrence probability of Head and Tail for each coin.

Program

```
if __name__ == '__main__':  
    count = int(input('Enter number of coins : '))  
    i = 0  
    l1 = []  
    while i < count:  
        f1 = input(f'Enter coin no - {i+1} result 1st :')  
        f2 = input(f'Enter coin no - {i+1} result 2nd :')  
        f3 = input(f'Enter coin no - {i+1} result 3rd :')  
        l1.append(list((i+1, f1, f2, f3)))  
        i = i + 1  
  
    i = 0  
    print(l1)  
  
    while i < count:  
  
        P_T = l1[i].count('t')  
        P_H = l1[i].count('h')
```

```

print(f'For coin - {i+1} probability of Tail is = {P_T / 3}')
print(f'For coin - {i+1} probability of Head is = {P_H / 3 }')

i = i + 1

```

Output

```

Enter number of coins : 2
Enter coin no - 1 result 1st :t
Enter coin no - 1 result 2nd :h
Enter coin no - 1 result 3rd :t
Enter coin no - 2 result 1st :h
Enter coin no - 2 result 2nd :t
Enter coin no - 2 result 3rd :h
[[1, 't', 'h', 't'], [2, 'h', 't', 'h']]
For coin - 1 probability of Tail is = 0.6666666666666666
For coin - 1 probability of Head is = 0.3333333333333333
For coin - 2 probability of Tail is = 0.3333333333333333
For coin - 2 probability of Head is = 0.6666666666666666

```

Rubrics

Problem Understating	Problem Analysis	Capability of writing program	Documentation	Total
02	03	04	01	10

Date: / /

Signature

PRACTICAL: 2**Date:** / /

TITLE: Find probability of throwing generic object like triangular pyramid, square cube, hexagonal cube etc.

- Accept no. of object
- Accept no. of trail for each object
- Accept each object trail value
- Display occurrence probability of each feature of each object.

Program

```
if __name__ == '__main__':

    object_count = int(input(f'Enter number of objects : '))

    i = 0

    j = 0

    object_list = []

    while i < object_count:

        trial_count = int(input(f'Enter number of trial for object {i + 1} : '))

        trial_list = []

        while j < trial_count:

            trial_list.append(input(f'For object {i + 1} Enter trial no = {j + 1} value :'))

            j = j + 1

        l2 = list((i, trial_list))

        object_list.append(l2)
```

```
j = 0

i = i + 1

i = 0

j = 0

print(object_list)

while i < object_count:

    temp = object_list[i][1]

    s1 = set(temp)

    l2 = list(s1)

    l2.sort()

    while j < len(s1):

        print(f'For OBJECT {i + 1} probability of value {l2[j]} is
{temp.count(l2[j]) / len(temp) }')

        j += 1

    j = 0

    i = i + 1
```

Output:

```

Enter number of objects : 2
Enter number of trial for object 1 : 3
For object 1 Enter trial no = 1 value :2
For object 1 Enter trial no = 2 value :1
For object 1 Enter trial no = 3 value :3
Enter number of trial for object 2 : 4
For object 2 Enter trial no = 1 value :3
For object 2 Enter trial no = 2 value :4
For object 2 Enter trial no = 3 value :2
For object 2 Enter trial no = 4 value :4
For object 2 Enter trial no = 5 value :3
For object 2 Enter trial no = 6 value :4
[[0, ['2', '1', '3']], [1, ['2', '6', '2', '4', '3', '4']]]
For OBJECT 1 probability of value 1 is 0.3333333333333333
For OBJECT 1 probability of value 2 is 0.3333333333333333
For OBJECT 1 probability of value 3 is 0.3333333333333333
For OBJECT 2 probability of value 2 is 0.3333333333333333
For OBJECT 2 probability of value 3 is 0.16666666666666666
For OBJECT 2 probability of value 4 is 0.3333333333333333
For OBJECT 2 probability of value 6 is 0.16666666666666666

```

Rubrics

Problem Understating	Problem Analysis	Capability of writing program	Documentation	Total
02	03	04	01	10

Date: / /

Signature

PRACTICAL: 3**Date:** / /**TITLE:** Find join probability and conditional probability.

- Accept no. of coins
- Accept trial count to toss all coins together (Event e).
- Find probability of event e1 in which we get at least on 'Head'
- Find probability of event e2 in which we get at least on 'Tail'
- Find joint probability of e1 and e2.
- $P(e1 \cap e2) = p(e1) - (e2) - P(e1 \cup e2)$

Program

```
if __name__ == '__main__':

    coin_count = int(input('Enter number of coins : '))

    # Tossing of multiple coins

    trial_count = int(input('Enter number of trials : '))

    i = 0

    l1 = []

    while i < trial_count:

        result = input(f'Enter all coins {i+1} trial result :')

        l1.append(list((i, result)))

        i = i + 1

    i = 0

    print(l1)

    # derive list of event e1 in which we get at least one 'H'
```

```
# derive list of event e2 in which we get at least one 'T'

e1 = []

e2 = []

while i < trial_count:

    if l1[i][1].__contains__('H'):

        e1.append(l1[i][1])

    if l1[i][1].__contains__('T'):

        e2.append(l1[i][1])

    i = i + 1

P_e1 = len(e1) / i

P_e2 = len(e2) / i

print(f'for event e1 probability at least one \'H\' is {len(e1)/i}')

print(f'for event e2 probability at least one \'T\' is {len(e2)/i}')

# e3 = e1 AND e2 intersect

e3 = []

for item in e1:

    if item in e2:

        e3.append(item)

print(e3)
```

```

P_e1_and_e2 = len(e3)/i

print(f'Joint probability e1 and e2 is is {len(e3)/i} ')

# Conditional probability :-> e1 is occur and probability of occurrence of e2

#  $P(e2/e1) = P(e2 \text{ and } e1) / P(e1)$ 

P_e2_given_e1 = P_e1_and_e2 / P_e1

print(f'Conditional probability e2 given e1 is {P_e2_given_e1} ')

P_e1_given_e2 = P_e1_and_e2 / P_e2

print(f'Conditional probability e1 given e2 is {P_e1_given_e2} ')

```

Output:

```

Enter number of coins : 3
Enter number of trials : 5
Enter all coins 1 trial result :HHT
Enter all coins 2 trial result :THT
Enter all coins 3 trial result :TTT
Enter all coins 4 trial result :TTH
Enter all coins 5 trial result :HTT
[[0, 'HHT'], [1, 'THT'], [2, 'TTT'], [3, 'TTH'], [4, 'HTT']]
for event e1 probability at least one 'H' is 0.8
for event e2 probability at least one 'T' is 1.0
['HHT', 'THT', 'TTH', 'HTT']
Joint probability e1 and e2 is is 0.8
Conditional probability e2 given e1 is 1.0
Conditional probability e1 given e2 is 0.8

```


Rubrics

Problem Understating	Problem Analysis	Capability of writing program	Documentation	Total
02	03	04	01	10

Date: / /**Signature**

PRACTICAL: 4**Date:** / /**TITLE:** Simulation of probability density function.

- **Graph-1:** Consider single coin tossing two times.

$S = \{H, T\} = \{1, 0\}$ ideal case.

Mean $\mu = 0.5$

Variance $\sigma^2 = 0.25$

Draw graph of Gaussian distribution for this values.

PDF $f(x) = 1/(2\pi \sigma) \cdot e^{(-1/2 * ((x - \mu) / \sigma)^2)}$

For simulation take $\sigma = \sigma/10 = 0.05$

- **Graph-2:** Consider single coin tossing multiple times.

$S = \{H, T, T, H, H, H, T, \dots\}$ practical case.

Calculate Mean $\mu = E(x) = \text{COUNT}('H') / \text{Total_Trial}$

Variance $\sigma^2 = E(x - \mu)^2 = \sum P(x) = \mu * (1 - \mu)$

Draw graph of Gaussian distribution for this values.

Program**Graph – 1:**

```
import math

from random import randint

import matplotlib.pyplot as plt

def print_hi(name):

    print(f'Hi, {name}')
```



```
if __name__ == '__main__':
```

```
trial_count = int(input('Enter number of trial : '))

trial_list = []

j = 0

while j < trial_count:

    n = randint(0, 100)

    n = n % 2

    trial_list.append(n)

    j = j + 1

print(trial_list)

mu1 = trial_list.count(1)/trial_count

sigma1 = pow((mu1 * (1.0 - mu1)), 0.5) / 10    # standard deviation

print(mu1, " ", sigma1)

x1 = []

f1 = []

l5 = [n/10 for n in range(0, 10, 1)]

for i1 in l5:

    x1.append(i1)

    temp = (i1 - mu1) / sigma1

    f1.append(((1 / (sigma1 * (2 * 3.14) ** 0.5)) * (math.e ** (-0.5 * temp *
temp))))
```

```

plt.plot(x1, f1, '*:g')

mu = 0.5

sigma = 0.05    # scaling by factor 10

x = []

f = []

l5 = [n/10 for n in range(0, 10, 1)] # l5 = [ 0.0,.1,.2,.3,.....,.9]

for i1 in l5:

    x.append(i1)

    temp = (i1 - mu) / sigma

    f.append((1 / (sigma * (2 * 3.14) ** 0.5)) * (math.e ** (-0.5 * temp * temp)))

plt.plot(x, f, '*:r')

plt.show()

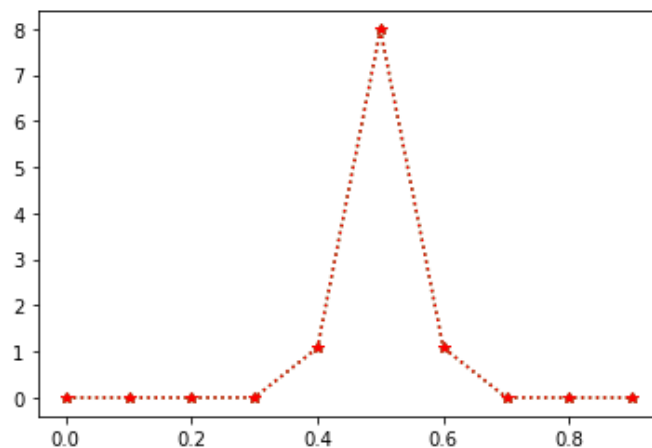
```

Output

```

Enter number of trial : 2
[0, 1]
0.5    0.05

```



Graph – 2:

```
import math

from random import randint

import matplotlib.pyplot as plt

def print_hi(name):

    print(f'Hi, {name}')
```



```
if __name__ == '__main__':

    trial_count = int(input('Enter number of trial : '))

    trial_list = []

    j = 0

    while j < trial_count:

        n = randint(0, 50000) + 17

        n = n % 4

        trial_list.append(n)

        j = j + 1

    print(trial_list)

    PR1 = trial_list.count(0) / trial_count

    PR2 = trial_list.count(1) / trial_count
```

```
PR3 = trial_list.count(2) / trial_count

PR4 = trial_list.count(3) / trial_count

mu = 0 * PR1 + 1 * PR2 + 2 * PR3 + 3 * PR4

sigma1 = ((pow((PR1 - mu), 2)+pow((PR2 - mu), 2)+pow((PR3 - mu), 2)+pow((PR4 -
mu), 2))*0.25) ** 0.5 / 10.0

print(mu, " ", sigma1)

x1 = []

f1 = []

l5 = [n/40 for n in range(1, 100, 1)]

for i1 in l5:

    x1.append(i1)

    temp = (i1 - mu) / sigma1

    f1.append((1 / (sigma1 * (2 * 3.14) ** 0.5)) * (math.e ** (-0.5 * temp *
temp)))

plt.plot(x1, f1, '*:g')

mu = 1.5

sigma = pow((1.5 * 1.5 + 0.25 + 0.25 + 1.5 * 1.5) / 4.0, 0.5) / 10.0

print(mu, " ", sigma)

x = []

f = []

l5 = [n/40 for n in range(1, 100, 1)]
```

```
for i1 in l5:

    x.append(i1)

    temp = (i1 - mu) / sigma

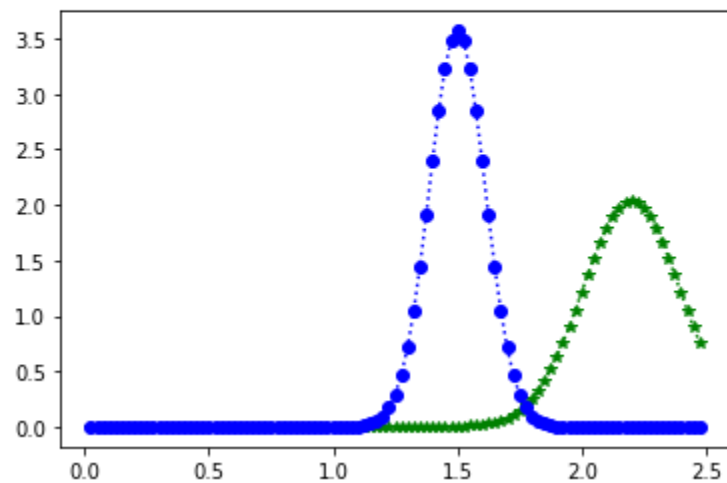
    f.append((1 / (sigma * (2 * 3.14) ** 0.5)) * (math.e ** (-0.5 * temp * temp)))

plt.plot(x, f, 'o:b')

plt.show()
```

Output

```
Enter number of trial : 5
[1, 2, 3, 2, 3]
2.2      0.19570385790780928
1.5      0.1118033988749895
```



Rubrics

Problem Understating	Problem Analysis	Capability of writing program	Documentation	Total
02	03	04	01	10

Date: / /**Signature**

PRACTICAL: 5**Date:** / /**TITLE:** Quadratic Equation Solution

Write a python program to find solution of equation like.....

$$x^{**2} + 7 * x + 12 = 0$$

$$5 * x + \cos(x) + 5 = 0$$

$$5 * x + 7 * y + 8 = 0 \quad , \quad 4 * x + 8 * y + z = 0 \quad \text{and} \quad 5 * y + 3 * z = 0$$

(Hint : Three variables , so three equation are given).

Program

(1) $x^{2} + 7 * x + 12 = 0$**

```
import numpy as np

from scipy.optimize import root

def func(X):

    return X**2+7*X+12

sol=root(func,0.3)

print(sol)
```

Output

```

fjac: array([[ -1.]])
fun: array([0.])
message: 'The solution converged.'
nfev: 12
qtf: array([-5.87583315e-11])
r: array([-1.00000047])
status: 1
success: True
x: array([-3.])

```

(2) $5 * x + \cos(x) + 5 = 0$

```

import numpy as np

from scipy.optimize import root

def func(X):

    return 5*X + np.cos(X) + 5

sol=root(func,0.3)

print(sol)

```

Output

```

fjac: array([[ -1.]])
fun: array([8.8817842e-16])
message: 'The solution converged.'
nfev: 8
qtf: array([-2.26307861e-12])
r: array([-5.88760577])
status: 1
success: True
x: array([-1.09212079])

```

(3) $5 * x + 7 * y + 8 = 0$, $4 * x + 8 * y + z = 0$ and $5 * y + 3 * z = 0$

```

import sympy as sympy

```

```
sympy.init_printing()

x,y,z=sympy.symbols('x,y,z')

c1=sympy.Symbol('c1')

f=sympy.Eq(5*x + 7*y + 8,0)

g=sympy.Eq(4*x + 8*y +z,0)

h=sympy.Eq(5*y + 3*z,0)

sympy.solve([f,g,h],(x,y,z))
```

Output

```
In [5]: import sympy as sympy
sympy.init_printing()
x,y,z=sympy.symbols('x,y,z')
c1=sympy.Symbol('c1')
f=sympy.Eq(5*x + 7*y + 8,0)
g=sympy.Eq(4*x + 8*y +z,0)
h=sympy.Eq(5*y + 3*z,0)
sympy.solve([f,g,h],(x,y,z))
```

```
Out[5]:  $\left\{ x : -\frac{152}{11}, y : \frac{96}{11}, z : -\frac{160}{11} \right\}$ 
```

Conclusion

- With python we can find the roots of a polynomial equation of degree 2 ($ax^2 + bx + c$) using the function `numpy.root`.
- SciPy has optimized and added functions that are frequently used in NumPy and Data Science.

Rubrics

Problem Understating	Problem Analysis	Capability of writing program	Documentation	Total
02	03	04	01	10

Date: / /**Signature**